Proyecto Semestral

Análisis y Diseño de Algoritmos

2026-B

UNIVERSIDAD DE CALDAS | UC

Índice general

1 Presentación y marco común 1

1.1 Presentación . . . 1
1.2 Qué significa «eficiente» en este proyecto . . . 1
1.3 Estructura general . . . 2

I Subproyectos 3

2 Subproyecto 1 — Configuración óptima de prompts bajo presupuesto de consultas (Proyecto Oráculo) 4

2.1 Escenario . . . 4
2.2 Modelo común y parámetros de la instancia base . . . 5
2.3 Parte 1 — Configuración óptima con oráculo exacto . . . 5
2.3.1 Algoritmo exacto: backtracking con poda por cota superior . . . 5
2.3.2 Reducción formal de NP-dureza (desde 3-SAT) . . . 6
2.3.3 Heurística: búsqueda local por coordenadas (voraz por categoría) . . . 6
2.4 Parte 2 — Oráculo ruidoso y presupuesto de consultas . . . 7
2.4.1 Algoritmo de referencia: reparto uniforme . . . 7
2.4.2 Eliminación sucesiva (*Successive Halving*) . . . 8
2.4.3 Comparación experimental exigida . . . 9
2.5 Variantes (todas obligatorias) . . . 9
2.6 Validación opcional con un modelo real . . . 9

3 Subproyecto 2 — Reparto de presupuesto y selección de participantes 10

3.1 Escenario . . . 10
3.2 Parte 1 — Reparto óptimo del presupuesto . . . 10
3.3 Parte 2 — Selección de nodos participantes . . . 11
3.4 Variantes (todas obligatorias) . . . 12

4 Subproyecto 3 — Ruta crítica con aceleración bajo presupuesto y cobertura mínima de auditoría 13

4.1 Escenario . . . 13
4.2 Parte 1 — Aceleración óptima de la cadena bajo presupuesto . . . 13
4.3 Parte 2 — Cobertura mínima de auditoría . . . 14
4.4 Variantes (todas obligatorias) . . . 15

5 Entregables, cronograma y evaluación 16

5.1 Entregables 16
5.2 Cronograma (13 semanas) 16
5.3 Criterios de evaluación 17
5.4 Notas finales 17

Sec 1.1 Presentación

Este documento formula tres proyectos de optimización algorítmica, uno por grupo de estudiantes, inspirados en el diseño real de un sistema de inteligencia artificial conversacional multiagente (un tipo de sistema cada vez más común en la industria), pero cada problema se plantea de forma abstracta y autocontenida, de modo que no depende de ninguna tecnología propietaria y puede formularse, resolverse y evaluarse íntegramente con las herramientas del curso.

Recuerden que no basta con programar algo que funcione, cada equipo debe demostrar formalmente qué tan eficiente es su solución, tanto para el caso exacto como para cualquier aproximación o heurística que proponga.

Sec 1.2 Qué significa «eficiente» en este proyecto

Antes de entrar a los tres problemas, esto es lo que se exige en cada uno (no es opcional ni queda a discreción del equipo):

- **Deben presentar:** la definición precisa del tamaño de entrada del problema, es decir, qué está creciendo cuando el problema se vuelve más grande, justificando por qué esa es la medida adecuada.
- **Deben demostrar:** la función de eficiencia $T(n)$ del algoritmo exacto, obtenida contando operaciones o resolviendo una recurrencia (no una estimación intuitiva, sino una deducción explícita, paso a paso, con su notación $O(\cdot)$ resultante).
- **Deben demostrar:** un argumento formal de por qué el problema es computacionalmente difícil, mediante una reducción en tiempo polinomial desde un problema NP-difícil ya conocido del curso (no una afirmación sin sustento).
- **Deben demostrar:** para cualquier algoritmo aproximado o heurístico, su propia función de eficiencia $T(n)$ y, cuando exista una garantía demostrable conocida (por ejemplo, una razón de aproximación), su demostración o su aplicación correcta citando el resultado del que proviene.
- **Deben demostrar:** una validación experimental, midiendo el tiempo real de ejecución para valores crecientes de n, graficando los resultados y contrastando el crecimiento observado contra el que predice la función $T(n)$ deducida teóricamente.

Un informe que solo reporte «tardó X segundos» sin la formalidad requerida no cumple el nivel del curso, sin importar qué tan bien funcione el código.

Sec 1.3 Estructura general

Cada grupo trabaja un problema, organizado en equipos de 2 personas. Los tres problemas comparten el mismo escenario de fondo (un sistema de asistentes de inteligencia artificial que debe tomar decisiones con recursos limitados), pero son independientes entre sí y pueden evaluarse por separado. Los tres tienen la misma estructura de dos partes, de forma que la carga de trabajo sea comparable.

Grupo	Problema	Técnica central	Tipo de dificultad
1	Configuración óptima de prompts: con oráculo exacto y con oráculo ruidoso bajo presupuesto de consultas	Backtracking con poda por cota + búsqueda local voraz; eliminación sucesiva (*Successive Halving*)	NP-difícil (reducción desde 3-SAT) + complejidad de muestreo (cotas de Hoeffding)
2	Reparto de un presupuesto limitado y selección de participantes	Algoritmo voraz óptimo (asignación cóncava) + aproximación submodular	Parte polinomial exacta + parte NP-difícil (reducción desde Cobertura Máxima)
3	Ruta crítica con aceleración bajo presupuesto, y cobertura mínima de auditoría	Programación dinámica (mochila) + aproximación voraz logarítmica	NP-difícil (reducción desde Mochila 0/1 y desde Cobertura de Vértices)

Tabla 1.1: Los tres subproyectos y su distribución por grupos.

Parte I

Subproyectos

Un problema por grupo: escenario, formalización, algoritmo exacto y aproximado con sus cotas, variantes obligatorias y su evaluación.

Subproyecto 1 — Configuración óptima de prompts bajo presupuesto de consultas (Proyecto Oráculo)

Parte I

Sec 2.1 Escenario

Un sistema conversacional debe responder solicitudes cumpliendo reglas de formato y restricciones estrictas de contenido. Para mejorar la tasa de éxito sin reentrenar el modelo, se antepone a cada solicitud un texto instruccional armado a partir de m categorías discretas (en la instancia base, $m = 5$: *Rol*, *Estrategia*, *Formato*, *Verificación* y *Recordatorio*), eligiendo exactamente una opción por categoría; cada combinación es una configuración, y el conjunto de todas ellas es el espacio C.

El problema tiene dos niveles, que se estudian por separado. Primeramente, aun si se conociera con exactitud qué reglas satisface cada opción, encontrar la mejor configuración es un problema combinatorio difícil (Parte 1). Progresivamente aparece la complicación práctica, ya que en la realidad esa información no se conoce y solo se puede estimar consultando al modelo, lo cual cuesta tiempo de GPU, así que se dispone de un presupuesto limitado de consultas ruidosas (Parte 2).

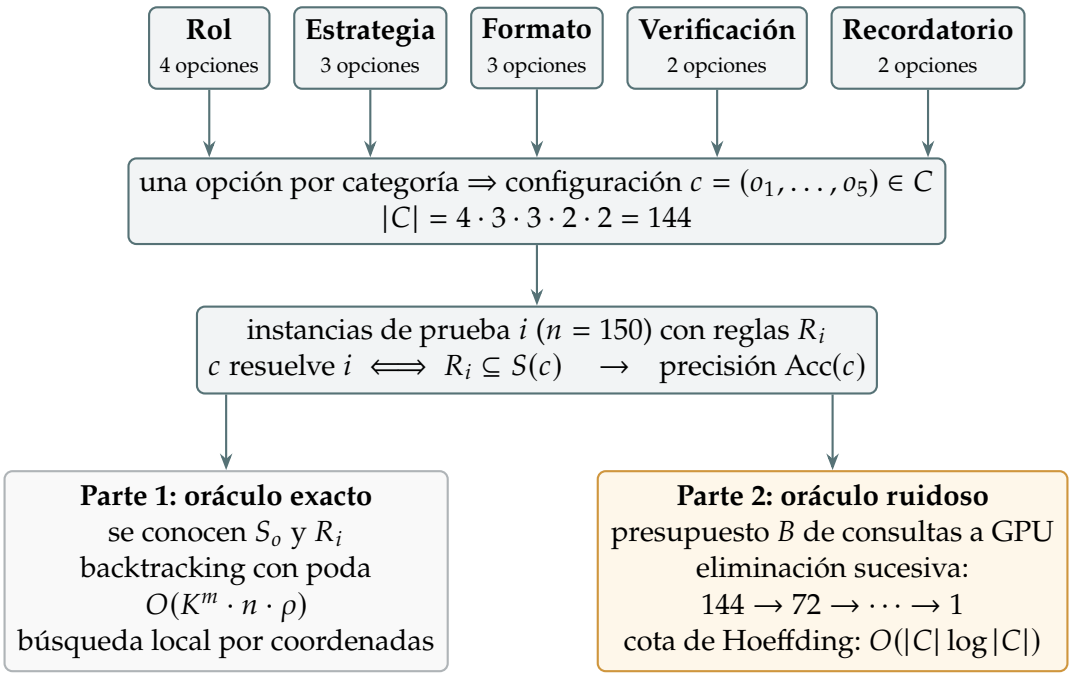


Figura 2.1: Estructura del Proyecto Oráculo: las cinco categorías generan el espacio C, las instancias imponen las reglas y las dos partes atacan el problema con oráculo exacto (Parte 1) y con consultas ruidosas bajo presupuesto (Parte 2).

Sec 2.2 Modelo común y parámetros de la instancia base

Definición 2.2.1 (Modelo común).

- ***Categorías y opciones:*** *hay m categorías, donde la categoría j tiene k_j opciones y $K = \max_j k_j$, de forma que $|C| = \prod_{j=1}^{m} k_j \le K^m$. En la instancia base se usa $k = (4, 3, 3, 2, 2)$, por lo que $|C| = 144$.*
- ***Reglas:*** *existe un universo de reglas R con $\rho = |R|$; cada opción o satisface un subconjunto $S_o \subseteq R$, y una configuración $c = (o_1, \dots, o_m)$ satisface $S(c) = \bigcup_{j=1}^{m} S_{o_j}$.*
- ***Instancias:*** *hay n instancias de prueba (en la base, $n = 150$), donde la instancia i exige un subconjunto de reglas $R_i \subseteq R$ y se considera resuelta por c si y solo si $R_i \subseteq S(c)$ (evaluación todo-o-nada).*
- ***Precisión:*** $\mathrm{Acc}(c) = \frac{1}{n}\left|\{\, i : R_i \subseteq S(c) \,\}\right|$.

Todas las instancias son sintéticas y las genera el propio equipo a partir de este modelo, de modo que el óptimo verdadero se puede conocer en instancias pequeñas.

Sec 2.3 Parte 1 — Configuración óptima con oráculo exacto

Definición 2.3.1 (Formalización con oráculo exacto). ***Tamaño de la entrada:*** *m (número de categorías), K (máximo de opciones por categoría), n (instancias) y ρ (reglas). Cabe mencionar que el tamaño de la entrada no es $|C|$, ya que C no se da listado sino implícitamente a través de las categorías, y precisamente por eso $|C|$ crece de forma exponencial en m.*
Entrada: *las opciones de cada categoría con sus conjuntos S_o y las instancias con sus conjuntos R_i.*
Salida: *una configuración $c^* \in C$.*
Objetivo: *maximizar* $\mathrm{Acc}(c^*)$.

2.3.1 Algoritmo exacto: backtracking con poda por cota superior

Algoritmo 2.3.1 (Backtracking con poda). *Se recorre el árbol de decisiones asignando una opción a cada categoría en orden. En un nodo interno donde ya se fijaron las categorías $1, \dots, j$ y se cubren las reglas P, sea $U_{>j}$ la unión de los S_o de* todas *las opciones de las categorías $j+1, \dots, m$; se define la cota*

$$\mathrm{UB} = \left|\{\, i : R_i \subseteq P \cup U_{>j} \,\}\right|,$$

y se corta la rama si UB *no supera el número de instancias resueltas por la mejor configuración encontrada hasta el momento.*

Se exige demostrar que la cota es válida, es decir, que ninguna completación de la rama resuelve más de UB instancias (la idea es que el conjunto de reglas de cualquier completación está contenido en $P \cup U_{>j}$).

Función de eficiencia: en el peor caso la poda no descarta nada, el árbol tiene $|C|$ hojas y a lo sumo $2|C|$ nodos (si todo $k_j \ge 2$), y en cada nodo se calcula la cota revisando las n instancias con un costo de $O(\rho)$ cada una (con conjuntos representados como vectores de bits), por lo que

$$T(m, K, n, \rho) = O(|C| \cdot n \cdot \rho) = O(K^m \cdot n \cdot \rho).$$

Se exige derivar formalmente esta cota (incluyendo la del número de nodos) y medir empíricamente cuántos nodos visita la poda frente a este peor caso exponencial en m.

2.3.2 Reducción formal de NP-dureza (desde 3-SAT)

Se define la versión de decisión Prompt-Dec: dadas las categorías, opciones, reglas e instancias y un entero t, ¿existe una configuración que resuelva al menos t instancias?

Dada una fórmula de 3-SAT con variables $x_1, \ldots, x_m$ y cláusulas $\varphi_1, \ldots, \varphi_n$, se construye:

- **Categorías:** una por variable (m categorías), cada una con dos opciones, *verdadero* y *falso* ($k_j = 2$).
- **Reglas:** una por cláusula, $R = \{r_1, \ldots, r_n\}$.
- **Instancias:** una por cláusula, donde la instancia i exige solo la regla r_i ($R_i = \{r_i\}$).
- **Cobertura de las opciones:** la opción *verdadero* de x_j satisface $\{\, r_i : x_j$ aparece en $\varphi_i \,\}$, y la opción *falso* satisface $\{\, r_i : \neg x_j$ aparece en $\varphi_i \,\}$.
- **Umbral:** $t = n$.

Proposición 2.3.1 (NP-dureza de Prompt-Dec)**.** *Existe una configuración que resuelve las n instancias si y solo si la fórmula de 3-SAT es satisfacible.*

Se exige demostrar que la construcción toma tiempo polinomial ($O(m \cdot n)$) y la equivalencia de la Proposición 2.3.1 (cada configuración corresponde a una asignación de verdad, y la instancia i se resuelve exactamente cuando la cláusula φ_i queda satisfecha). Como 3-SAT es NP-completo, Prompt-Dec es NP-difícil y por lo tanto también lo es la versión de optimización.

2.3.3 Heurística: búsqueda local por coordenadas (voraz por categoría)

Algoritmo 2.3.2 (Búsqueda local por coordenadas)**.** *Partiendo de una configuración inicial, se recorre cada categoría en orden, se fijan las otras $m - 1$ y se evalúan todas las opciones de la categoría actual, quedándose con la mejor; una pasada completa recorre las m categorías y el proceso se repite hasta que una pasada no mejore nada (convergencia) o hasta I pasadas.*

Función de eficiencia: una pasada evalúa $\sum_j k_j \leq mK$ configuraciones, cada una con costo $O(n \cdot \rho)$, por lo que

$$T(m, K, n, \rho, I) = O(I \cdot m \cdot K \cdot n \cdot \rho),$$

que es polinomial, a diferencia del exacto.

Teorema 2.3.1 (Garantía en el caso de una regla por instancia)**.** *Si $|R_i| = 1$ para toda instancia (que es justamente el caso de la reducción anterior), el número de instancias resueltas es una función de cobertura, monótona y submodular sobre el conjunto de opciones elegidas, y elegir una opción por categoría es una restricción de matroide de partición. En ese caso, una sola pasada voraz partiendo de ninguna opción elegida garantiza al menos la mitad del óptimo (Fisher et al., 1978).*

Resultado que el equipo debe citar correctamente, aplicar y verificar en sus instancias pequeñas comparando contra el exacto.

Sin garantía en el caso general: cuando una instancia exige varias reglas, la función deja de ser submodular y la búsqueda local puede quedar atrapada en óptimos locales arbitrariamente malos. Se exige analizar el siguiente contraejemplo y generalizarlo:

Ejemplo 2.1 (Un contraejemplo para la búsqueda local). *Hay dos categorías, A con opciones A_1, A_2 y B con opciones B_1, B_2, donde cada opción satisface una regla propia (a_1, a_2, b_1, b_2 respectivamente). Una instancia exige $\{a_1, b_1\}$ y otras q instancias exigen $\{a_2, b_2\}$. Partiendo de (A_1, B_1) se resuelve una instancia, y cambiar una sola coordenada ((A_2, B_1) o (A_1, B_2)) no resuelve ninguna, así que la búsqueda local se detiene con valor 1 mientras que el óptimo (A_2, B_2) vale q, y la razón $1/q$ se puede hacer tan mala como se quiera.*

Sec 2.4 Parte 2 — Oráculo ruidoso y presupuesto de consultas

Escenario: en la práctica no se conocen los S_o ni los R_i; lo único que se puede hacer es consultar al modelo con una configuración c sobre una instancia elegida al azar y observar si la respuesta fue correcta (1) o no (0). En el simulador sintético, cada consulta de c devuelve 1 con probabilidad p_c (obtenida del modelo de la Parte 1 más un ruido de generación, por ejemplo $p_c = (1 - \varepsilon)\,\mathrm{Acc}(c) + \delta\,(1 - \mathrm{Acc}(c))$ con ε, δ pequeños).

Definición 2.4.1 (Formalización con oráculo ruidoso). ***Tamaño de la entrada:*** *$|C|$ (aquí sí, ya que las configuraciones candidatas se tratan como una lista explícita de «brazos»), el presupuesto B y el número de instancias n.*
Entrada: *el conjunto C, acceso al oráculo de consultas y un presupuesto B.*
Salida: *una asignación de m_c consultas a cada configuración, con $\sum_{c \in C} m_c \le B$, y una configuración elegida $\hat{c}$.*
Objetivo: *maximizar la probabilidad de que $\hat{c}$ sea la mejor configuración c^* (o, equivalentemente, minimizar el arrepentimiento simple $p_{c^*} - p_{\hat{c}}$).*

Observación. *Se exige explicar en el informe por qué esta parte **no** se analiza con NP-dureza, ya que con C explícito el costo de cómputo es polinomial y el recurso escaso son las consultas, de forma que la dificultad es de complejidad de muestreo (cuántas consultas se necesitan para distinguir configuraciones cuyas precisiones difieren poco). La NP-dureza del proyecto es la de la Parte 1.*

Sea $\Delta_c = p_{c^*} - p_c$ la brecha de cada configuración respecto a la mejor.

2.4.1 Algoritmo de referencia: reparto uniforme

Algoritmo 2.4.1 (Reparto uniforme). *Se asignan $m = \lfloor B/|C| \rfloor$ consultas a cada configuración, se estima $\hat{p}_c$ como la proporción de aciertos y se elige la de mayor estimación.*

Función de eficiencia: $T(|C|, B) = O(B + |C|)$ en cómputo, con exactamente $|C| \cdot m \le B$ consultas.

Proposición 2.4.1 (Cota de error del reparto uniforme). *Usando la desigualdad de Hoeffding sobre la diferencia $\hat{p}_c - \hat{p}_{c^*}$ (que es un promedio de $2m$ variables independientes acotadas) y la cota de la unión,*

$$\Pr[\hat{c} \neq c^*] \;\le\; \sum_{c \neq c^*} e^{-m\,\Delta_c^2}.$$

Se exige derivar la Proposición 2.4.1. **Limitación analítica:** si m es pequeño la varianza es alta, y debido a la evaluación todo-o-nada muchas configuraciones buenas aparecen con 0 % de aciertos por puro ruido; además se gastan las mismas consultas en configuraciones claramente malas que en las competitivas.

2.4.2 Eliminación sucesiva (*Successive Halving*)

Algoritmo 2.4.2 (Eliminación sucesiva). *Se organiza la búsqueda en rondas $r = 0, 1, \ldots,$ donde en la ronda r cada configuración sobreviviente se evalúa con $n_r = n_0 \cdot 2^r$ instancias distintas elegidas al azar, y al final de la ronda se conserva la mejor mitad, $|C_{r+1}| = \lfloor |C_r|/2 \rfloor$ (con un mínimo de 1), hasta que queda una sola configuración.*

Número de rondas: partiendo de $|C|$ y dividiendo con piso, se llega a una configuración después de $R = \lfloor \log_2 |C| \rfloor$ rondas evaluadas, lo cual se exige demostrar.

Condición de factibilidad: como no se pueden usar más instancias de las que existen, se exige $n_0 \cdot 2^{R-1} \leq n$; en caso contrario se debe usar $n_r = \min(n_0 \cdot 2^r, n)$ y rehacer el análisis.

Función de eficiencia (en consultas): como $|C_r| \leq |C|/2^r$, cada ronda consume a lo sumo $|C|/2^r \cdot n_0 \cdot 2^r = |C| \cdot n_0$ consultas, por lo que

$$T(|C|, n_0) \;\leq\; |C| \cdot n_0 \cdot \lfloor \log_2 |C| \rfloor = O(|C| \log |C|),$$

que no depende de n siempre que se cumpla la condición de factibilidad.

Cálculo exigido para la instancia base: con $|C| = 144$, $n_0 = 1$ y $n = 150$ se tienen $R = 7$ rondas y la última usa $n_6 = 64 \leq 150$ instancias, con el siguiente consumo, que se exige reproducir y contrastar con la cota ($144 \cdot 1 \cdot 7 = 1008$) y con la evaluación exhaustiva de todas las configuraciones sobre todas las instancias ($144 \cdot 150 = 21\,600$ consultas).

Ronda r	$\lvert C_r \rvert$	n_r	Consultas
0	144	1	144
1	72	2	144
2	36	4	144
3	18	8	144
4	9	16	144
5	4	32	128
6	2	64	128
Total			**976**

Tabla 2.1: Consumo de consultas de la eliminación sucesiva para $|C| = 144$, $n_0 = 1$ y $n = 150$.

Teorema 2.4.1 (Garantía de *Sequential Halving*). *La versión de presupuesto fijo del algoritmo identifica la mejor configuración con probabilidad de error acotada por una expresión del orden de $\log_2 |C| \cdot \exp\big(-B/(8 H_2 \log_2 |C|)\big)$, con $H_2 = \max_i i/\Delta^2_{(i)}$ calculado sobre las brechas ordenadas (Karnin et al., 2013). Se exige comparar esta cota con la del reparto uniforme para el mismo B.*

2.4.3 Comparación experimental exigida

Se debe demostrar con datos y gráficas cuál de los esquemas aprovecha mejor un mismo presupuesto B, graficando para varios valores de B la frecuencia con que cada algoritmo identifica c^* y el arrepentimiento simple promedio, sobre al menos 100 repeticiones con semillas distintas, y contrastando el comportamiento observado contra las cotas teóricas.

Sec 2.5 Variantes (todas obligatorias)

- **Espacio extendido con temperatura:** se incorpora la temperatura de generación como una sexta categoría con 3 niveles, de forma que $|C| = 432$, y se exige recalcular el número de rondas, la condición de factibilidad y el consumo de la eliminación sucesiva, así como medir el efecto sobre el backtracking de la Parte 1.
- **Asignación adaptativa por bandidos multi-brazo (UCB1):** se formula la asignación de consultas como un bandido multi-brazo usando el índice de confianza superior de Auer et al. (2002), derivando su costo de cómputo (el índice de todas las configuraciones cambia en cada paso, por lo que la implementación directa cuesta $O(B \cdot |C|)$) y discutiendo que su garantía clásica es sobre el arrepentimiento acumulado, no sobre la identificación de la mejor configuración.
- **Metaheurística (recocido simulado o algoritmo genético):** se formulan operadores de mutación y cruce sobre la tupla de opciones y se evalúa experimentalmente su convergencia en la Parte 1 frente a la búsqueda local y al exacto, incluyendo instancias construidas con la estructura del contraejemplo del Ejemplo 2.1.

Sec 2.6 Validación opcional con un modelo real

De manera opcional (y sin peso en la nota), el equipo puede repetir la comparación de la Parte 2 con un modelo de lenguaje real en Colab sobre una instancia pequeña, reportando el costo real en consultas y tiempo de GPU; el análisis formal y la experimentación exigidos se hacen siempre sobre el simulador sintético.

Sec 3.1 Escenario

Una red de N nodos entrena de forma colaborativa un modelo a lo largo de T rondas, sin compartir datos crudos entre ellos, protegiendo cada actualización con un mecanismo que introduce ruido calibrado (más ruido protege más, pero reduce la utilidad de la actualización). Cada nodo tiene un presupuesto total de «ruido tolerable» a lo largo de todo el proyecto, y en cada ronda hay que decidir qué subconjunto de nodos participa y cuánto de su presupuesto se gasta.

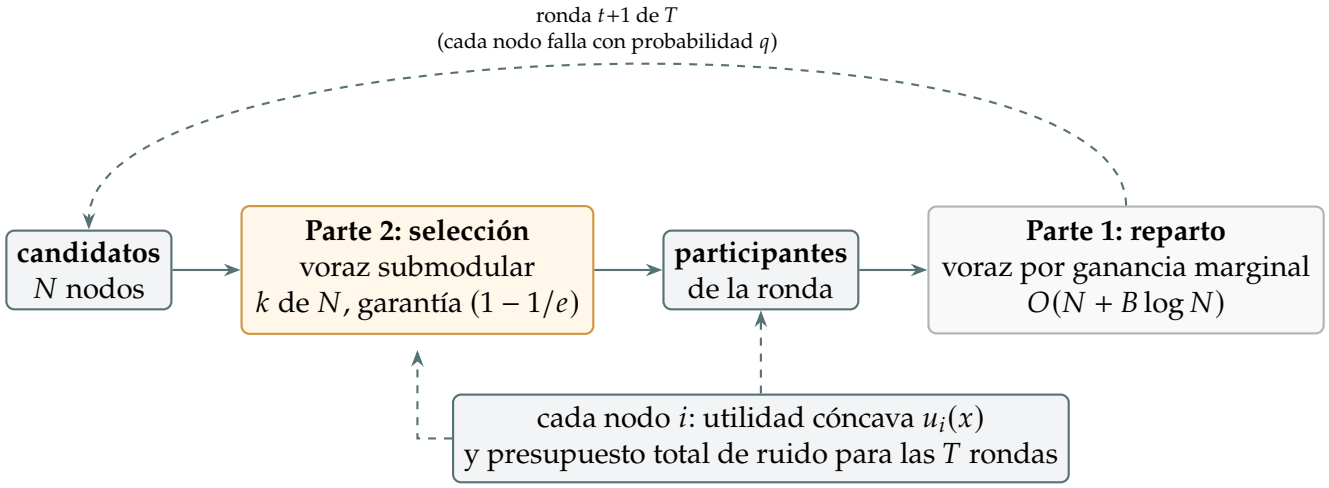


Figura 3.1: Una ronda del entrenamiento colaborativo: la Parte 2 selecciona los k participantes y la Parte 1 reparte entre ellos el presupuesto B; el proceso se repite durante T rondas con fallas.

Sec 3.2 Parte 1 — Reparto óptimo del presupuesto

Definición 3.2.1 (Reparto óptimo del presupuesto)**.** ***Tamaño de la entrada:*** *N, el número de nodos ya seleccionados para participar en la ronda, y B, el presupuesto total (medido en unidades enteras).*
Entrada: *para cada nodo, una función de utilidad $u_i(x)$ definida sobre enteros no negativos, creciente y cóncava (cada unidad adicional de presupuesto aporta una ganancia marginal $u_i(x+1) - u_i(x)$ no mayor que la anterior), y un presupuesto total B.*
Objetivo: *repartir B entre los N nodos, con $\sum_i x_i \le B$, para maximizar la utilidad total $\sum_i u_i(x_i)$.*

Este problema sí tiene solución exacta en tiempo polinomial en N, y se exige demostrarlo.

Algoritmo 3.2.1 (Voraz por ganancia marginal)**.** *En cada paso, el algoritmo entrega la siguiente unidad de presupuesto al nodo cuya ganancia marginal es mayor en ese momento, hasta agotar las B unidades.*

Teorema 3.2.1 (Optimalidad del voraz)**.** *El voraz por ganancia marginal devuelve un reparto de utilidad total máxima.*

Demostración. Por un argumento de intercambio: si una solución no reparte las unidades en orden de ganancia marginal decreciente, siempre es posible mover una unidad del nodo con menor ganancia marginal (en su última unidad) al nodo con mayor ganancia marginal (en su siguiente unidad) sin empeorar la utilidad total, lo cual, usando la concavidad, lleva a que la solución voraz es al menos tan buena como cualquier óptima. ■

Función de eficiencia: usando una cola de prioridad (montículo) con las ganancias marginales, construir el montículo cuesta $O(N)$ y cada una de las B unidades se asigna en $O(\log N)$, por lo que

$$T(N, B) = O(N + B \log N).$$

Se exige explicar que esta cota es **pseudo-polinomial**, ya que B aparece como valor numérico y no como longitud de su representación en bits (el mismo fenómeno que en la mochila del Subproyecto 4), y discutir cómo cambiaría el análisis si el presupuesto fuera continuo (reparto por nivelación, *water-filling*). Esta parte no se debe resolver por fuerza bruta ni por metaheurísticas.

Sec 3.3 Parte 2 — Selección de nodos participantes

Definición 3.3.1 (Selección de participantes)**.** ***Tamaño de la entrada:*** *N, el número total de nodos candidatos, y k, el número de nodos que se pueden seleccionar en la ronda ($k < N$).*
Entrada: *una función de utilidad conjunta f que, dado un subconjunto de nodos, reporta la utilidad total obtenida, y que es **monótona** (agregar nodos nunca reduce la utilidad) y **submodular** (agregar un nodo nuevo a un subconjunto grande aporta menos utilidad marginal que agregarlo a uno pequeño).*
Objetivo: *elegir el subconjunto de tamaño k que maximiza la utilidad conjunta.*

Proposición 3.3.1 (NP-dureza, reducción desde Cobertura Máxima)**.** *Dada una instancia de Cobertura Máxima (un universo U, una colección de subconjuntos $S_1, \ldots, S_N$ y enteros k y t, con la pregunta de si existen k subconjuntos cuya unión tenga al menos t elementos), se construye un nodo por subconjunto y la utilidad $f(A) = \left|\bigcup_{i \in A} S_i\right|$, que es monótona y submodular; existe una selección de k nodos con utilidad al menos t si y solo si la instancia de Cobertura Máxima es positiva.*

Se exige presentar la reducción completa de la Proposición 3.3.1. Como Cobertura Máxima es NP-difícil (por reducción conocida desde Cobertura de Conjuntos), el problema de selección también lo es.

Teorema 3.3.1 (Cota del voraz submodular)**.** *El algoritmo voraz que, en cada uno de los k pasos, agrega el nodo de mayor ganancia marginal garantiza una utilidad de al menos $(1 - 1/e) \approx 63\,\%$ de la óptima cuando f es monótona y submodular (Nemhauser et al., 1978).*

Resultado clásico que el equipo debe citar correctamente y aplicar. Como verificar esa cota requiere conocer el óptimo, se exige calcularlo por fuerza bruta ($\binom{N}{k}$ subconjuntos) en instancias pequeñas y comprobar que el voraz la respeta.

Función de eficiencia del algoritmo voraz: en cada uno de los k pasos se evalúa la ganancia marginal de cada uno de los nodos restantes, lo que da $O(N \cdot k)$ evaluaciones de f; si una evaluación cuesta τ, entonces $T(N, k) = O(N \cdot k \cdot \tau)$, y se exige precisar τ para la función de utilidad que el equipo implemente.

Sec 3.4 Variantes (todas obligatorias)

- **Nodos con costos de participación distintos:** cada nodo tiene un costo y hay un presupuesto de participación en lugar de un cupo fijo k (cobertura máxima con presupuesto, *budgeted maximum coverage*). El voraz simple ya no garantiza $(1-1/e)$, por lo que se exige implementar y citar correctamente la versión de Khuller et al. (1999), que toma el mejor entre el voraz por razón utilidad/costo y el mejor nodo individual (con garantía $\frac{1}{2}(1-1/e)$), o bien su versión con enumeración parcial (con garantía $1-1/e$), derivando la función de eficiencia de la versión elegida.
- **Repetición en T rondas con fallas:** en cada ronda cada nodo falla de forma independiente con probabilidad q y no puede participar, y el presupuesto de cada nodo es total para las T rondas; se exige derivar el costo total $O\big(T \cdot (Nk\tau + N + B \log N)\big)$ y comparar experimentalmente al menos dos políticas de gasto del presupuesto a lo largo de las rondas (uniforme frente a concentrado al inicio).
- **Nodos relacionados entre sí:** los nodos forman un grafo G y el presupuesto asignado a un nodo afecta a sus vecinos, en dos casos. En el *caso aditivo*, la utilidad de i es $u_i(x_i) + \gamma \sum_{j \in N(i)} h(x_j)$ con h cóncava y creciente, y se exige demostrar que la utilidad total sigue siendo separable y cóncava por nodo (agrupando términos), de forma que el voraz de la Parte 1 sigue siendo óptimo. En el *caso de competencia*, la utilidad de i se reduce según el presupuesto de sus vecinos (por ejemplo $u_i(x_i)\,\big(1 - \gamma \sum_{j \in N(i)} x_j / B\big)$), y se exige analizar si el voraz sigue siendo óptimo, dando un contraejemplo si no lo es y proponiendo una heurística con su función de eficiencia.

Subproyecto 3 — Ruta crítica con aceleración bajo presupuesto y cobertura mínima de auditoría

Parte I

Sec 4.1 Escenario

Un flujo de trabajo procesa cada solicitud a través de una cadena de n etapas dispuestas en serie. Cada etapa i tiene una duración normal t_i y puede acelerarse a un costo c_i, reduciendo su duración en d_i unidades de tiempo (con $0 \le d_i \le t_i$), y hay un presupuesto total B para acelerar etapas. Por separado, cada solicitud que atraviesa el flujo puede necesitar una revisión de cumplimiento normativo a fondo, y como revisar todas es demasiado costoso, solo se audita un subconjunto.

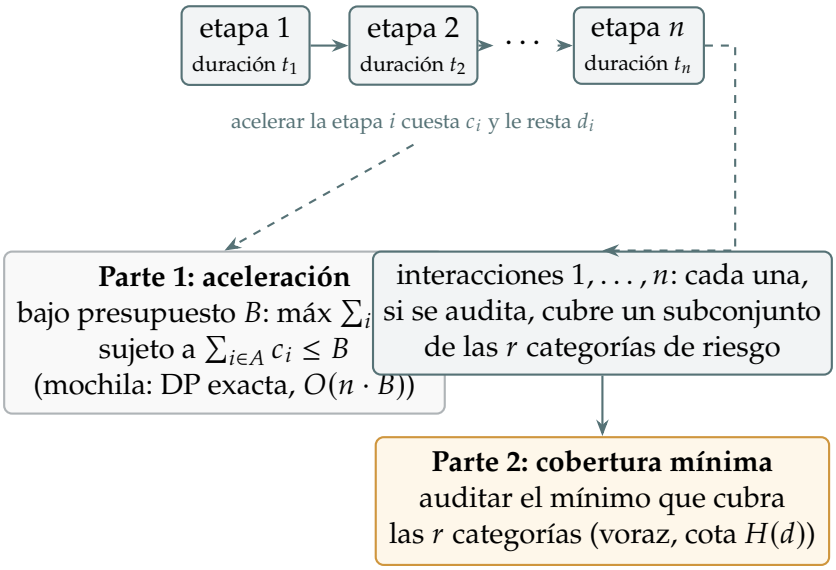


Figura 4.1: Las dos partes del Subproyecto 3: acelerar etapas de la cadena bajo presupuesto B (mochila) y elegir la auditoría mínima que cubra todas las categorías de riesgo (cobertura de conjuntos).

Sec 4.2 Parte 1 — Aceleración óptima de la cadena bajo presupuesto

Tamaño de la entrada: n, el número de etapas de la cadena, y B, el presupuesto disponible (entero).

Objetivo: elegir qué subconjunto de etapas acelerar, sin exceder el presupuesto B, para minimizar la duración total de la cadena, que es la suma de las duraciones de todas las etapas (aceleradas o no), ya que al ser una cadena en serie todas están siempre en el camino crítico. Como la duración total es $\sum_i t_i - \sum_{i \in A} d_i$ para el conjunto acelerado A, minimizarla equivale a maximizar $\sum_{i \in A} d_i$ sujeto a $\sum_{i \in A} c_i \le B$.

Proposición 4.2.1 (NP-dureza, reducción desde Mochila 0/1)**.** *Dada una instancia de decisión de Mochila 0/1 (objetos con peso w_i y valor v_i, capacidad W y umbral V), se construye una etapa por objeto con $c_i = w_i$, $d_i = v_i$ y $t_i = v_i$ (lo que garantiza $d_i \le t_i$), presupuesto $B = W$ y la pregunta de si se puede lograr una duración total de a lo sumo $\sum_i v_i - V$; la respuesta es afirmativa si y solo si la mochila admite valor al menos V.*

Se exige presentar la reducción en la dirección correcta de la Proposición 4.2.1. Como Mochila 0/1 es NP-difícil (ya demostrado en el curso), la aceleración de la cadena también lo es, y como una cadena en serie es un caso particular de un flujo general (con etapas en paralelo y rutas alternativas), el problema general es al menos igual de difícil.

Algoritmo 4.2.1 (Programación dinámica)**.** *El problema se resuelve de forma exacta con programación dinámica sobre una tabla indexada por etapa y presupuesto usado, con la recurrencia*

$$\mathrm{DP}[i][b] = \begin{cases} \max\left(\mathrm{DP}[i-1][b],\ \mathrm{DP}[i-1][b-c_i] + d_i\right) & \text{si } c_i \le b, \\ \mathrm{DP}[i-1][b] & \text{si } c_i > b, \end{cases}$$

con $\mathrm{DP}[0][b] = 0$.

La tabla tiene $(n+1) \times (B+1)$ celdas y cada una se calcula en tiempo constante, así que $T(n, B) = O(n \cdot B)$, más $O(n)$ para reconstruir el conjunto de etapas aceleradas recorriendo la tabla hacia atrás. Se exige explicar y justificar que esta cota es **pseudo-polinomial**, eficiente cuando B es un número pequeño pero exponencial en el tamaño de la representación de B en bits, y que esto no contradice la NP-dureza (el problema es NP-difícil en sentido débil).

Sec 4.3 Parte 2 — Cobertura mínima de auditoría

Definición 4.3.1 (Cobertura mínima de auditoría)**.** ***Tamaño de la entrada:*** *n, el número de interacciones candidatas a auditar, y r, el número de categorías de riesgo distintas (tipos de sesgo, fuga de datos personales, fallas de formato, entre otras).*
Entrada: *para cada interacción, el subconjunto de categorías de riesgo que cubriría si se audita.*
Objetivo: *elegir el menor número posible de interacciones a auditar de modo que, entre todas las elegidas, se cubra cada una de las r categorías al menos una vez (se asume que la unión de todas las interacciones cubre las r categorías, y se exige verificar esta condición de factibilidad en $O(n \cdot r)$ antes de ejecutar cualquier algoritmo).*

Proposición 4.3.1 (NP-dureza, reducción desde Cobertura de Vértices)**.** *Este es el problema clásico de Cobertura de Conjuntos (*Set Cover*), NP-completo en su versión de decisión (Karp, 1972). Dado un grafo $G = (V, E)$ y un entero k, las categorías de riesgo son las aristas, cada vértice es una interacción que cubre las aristas incidentes a él, y existe una cobertura de vértices de tamaño a lo sumo k si y solo si existe una auditoría de a lo sumo k interacciones que cubra todas las categorías.*

Se exige presentar la reducción estándar de la Proposición 4.3.1.

Teorema 4.3.1 (Cota del voraz para cobertura)**.** *El algoritmo voraz que en cada paso elige la interacción que cubre más categorías todavía no cubiertas nunca usa más de $H(d) = 1 + \frac{1}{2} + \cdots + \frac{1}{d}$ veces el número mínimo real de interacciones, donde d es el máximo de categorías que cubre una sola interacción, y como $d \le r$ se tiene $H(d) \le H_r \approx \ln r + 1$ (Johnson, 1974; Chvátal, 1979).*

Se exige citar el resultado correctamente y verificarlo empíricamente comparando contra el óptimo calculado por fuerza bruta en instancias pequeñas.

Función de eficiencia del algoritmo voraz: cada elección cubre al menos una categoría nueva y elige una interacción distinta, por lo que hay a lo sumo $\min(n, r)$ elecciones, y cada una revisa las n interacciones comparando cuántas categorías nuevas cubre cada una, con un costo de $O(n \cdot r)$ por elección, dando

$$T(n, r) = O\big(n \cdot r \cdot \min(n, r)\big)$$

en el peor caso.

Sec 4.4 Variantes (todas obligatorias)

- **Cobertura ponderada por costo de auditoría:** auditar la interacción j tiene un costo a_j (por ejemplo, según su longitud), y se busca cubrir todas las categorías con costo total mínimo (Cobertura de Conjuntos ponderada). El voraz elige en cada paso la interacción de menor costo por categoría nueva cubierta, y se exige demostrar o citar correctamente que la cota $H(d)$ se conserva (Chvátal, 1979) y derivar su función de eficiencia.

- **Combinación de las dos partes bajo un presupuesto conjunto:** hay un único presupuesto B para acelerar etapas y auditar interacciones, la auditoría debe cubrir todas las categorías y lo que sobre se usa para acelerar. Se exige mostrar que la solución exacta consiste en resolver la cobertura ponderada de costo mínimo a^* y luego la programación dinámica con presupuesto $B - a^*$, que usar el voraz de cobertura da una solución factible pero con costo de auditoría de hasta $H(d) \cdot a^*$, y analizar (teórica y experimentalmente) cuánta reducción de tiempo se pierde por ese exceso, aprovechando que la última fila de la tabla DP$[n][\cdot]$ ya contiene la mejor reducción para todos los presupuestos posibles.

- **Etapas en paralelo (grafo acíclico dirigido):** el flujo es un grafo acíclico dirigido general con n etapas y m aristas de precedencia, y la duración del flujo es la de la ruta más larga, que se calcula con programación dinámica sobre el orden topológico en $T(n, m) = O(n + m)$ (se exige deducirla). Cabe mencionar que decidir qué acelerar ya no es una mochila, ya que al acelerar etapas la ruta crítica cambia, y el problema resultante (compromiso tiempo-costo discreto) es fuertemente NP-difícil (De et al., 1997), por lo que la programación dinámica pseudo-polinomial no se extiende. Se exige implementar la solución exacta por enumeración de subconjuntos para instancias pequeñas, en $O(2^n(n + m))$, y una heurística que en cada paso recalcula la ruta crítica y acelera, entre las etapas de esa ruta que caben en el presupuesto restante, la de mejor razón d_i/c_i, con costo $O(n \cdot (n + m))$, comparando ambas experimentalmente y mostrando una instancia donde la heurística no es óptima.

Entregables, cronograma y evaluación



Parte I

Sec 5.1 Entregables

- **Informe técnico:** incluye una sección de análisis de complejidad que no puede omitirse, con la definición del tamaño de la entrada, la deducción de $T(n)$ para el algoritmo exacto, la prueba de NP-dureza por reducción, el algoritmo aproximado o heurístico con su cota (demostrada o correctamente citada y aplicada) y la validación experimental que compare el crecimiento medido contra el crecimiento teórico.
- **Implementación:** código reproducible (con semillas fijas) que genere instancias sintéticas de tamaño creciente, con al menos 5 tamaños distintos en un rango suficiente para observar el crecimiento asintótico, mida tiempos y grafique el tiempo contra el tamaño de la entrada.
- **Sustentación:** de 15 a 20 minutos, donde el equipo debe poder explicar y defender, sin ayuda de notas, la reducción de NP-dureza de su problema y la deducción de la función de eficiencia de su algoritmo exacto.

Sec 5.2 Cronograma (13 semanas)

Semanas	Hito
1 – 2	Formación de equipos; entrega escrita de la formalización del problema (tamaño de la entrada, entrada, salida y objetivo de ambas partes), sin código todavía
3 – 4	Prueba formal de NP-dureza por reducción (entrega parcial, revisada por el docente antes de continuar)
5 – 6	Algoritmo exacto (backtracking, fuerza bruta o programación dinámica) y deducción formal de su $T(n)$
7 – 8	Algoritmo aproximado o heurístico, su función de eficiencia y su cota de aproximación cuando exista
9 – 11	Experimentación y variantes: generación de instancias, medición de tiempos y comparación contra la función teórica
12 – 13	Entrega final y sustentaciones

Tabla 5.1: Cronograma del semestre.

Componente	Peso
Formalización del problema (tamaño de la entrada, entrada, salida, objetivo)	10 %
Prueba de NP-dureza por reducción, correcta y completa	10 %
Deducción formal de la función de eficiencia del algoritmo exacto	10 %
Algoritmo aproximado o heurístico: diseño, función de eficiencia y cota de aproximación	15 %
Validación experimental (medición y comparación contra la teoría)	10 %
Sustentación	45 %

Tabla 5.2: Criterios de evaluación y pesos.

Sec 5.3 Criterios de evaluación

Sec 5.4 Notas finales

Los tres problemas son independientes en su implementación, pero pueden cerrarse en una sesión conjunta donde los tres grupos presenten sus resultados como decisiones distintas dentro de un mismo sistema mayor. Ningún subproyecto requiere datos reales de ningún proyecto externo, todas las instancias son sintéticas y las generan los propios estudiantes según los parámetros descritos en cada sección (la validación con un modelo real del Subproyecto 2 es opcional y no tiene peso en la nota). El criterio de éxito no es «que el código corra», sino que cada equipo pueda, al final del semestre, sostener con rigor por qué su solución es eficiente, y qué tan eficiente es exactamente, en el lenguaje formal del curso.

Bibliografía

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

Vasek Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.

Prabuddha De, Elizabeth J. Dunne, Jay B. Ghosh, and Craig E. Wells. Complexity of the discrete time-cost tradeoff problem for project networks. *Operations Research*, 45(2):302–306, 1997.

Marshall L. Fisher, George L. Nemhauser, and Laurence A. Wolsey. An analysis of approximations for maximizing submodular set functions—II. *Mathematical Programming Study*, 8:73–87, 1978.

David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.

Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1238–1246, 2013.

Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York, 1972.

Samir Khuller, Anna Moss, and Joseph Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.

George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.